Stop configuring a system.
Start briefing an investigator.

Teaching Ray a new investigation used to mean a five-step wizard full of forms: pick a template, upload a policy, fill in step name, owner and SLA, review a flow chart, apply. Every one of those screens asks the user to translate what they already know into the product's structure. **A conversation removes the translation.** You say the job in your own words; Ray writes the structure and shows it back for you to read, correct and sign.

~~Rule builder & forms~~	→	**One conversation**
~~A list of cases~~	→	**A board of decisions**
~~Read the case file~~	→	**Watch it happen**
~~Configuration screens~~	→	**A playbook in plain English**
~~One fixed layout~~	→	**A workspace analysts shape**

HOW WE GOT HERE

Three generations of the same idea

Each version kept what worked and removed the thing that was still getting in the user's way.

v19 THE PRODUCT TODAY

A wizard with forms

Five numbered steps — choose a template, upload the policy, review the drafted workflow, define the report, apply. Inside them: 22 input fields, 12 dropdowns, step name / owned by / SLA, and tables of versions, comments and suggestions.

⚠ Ray is a **button inside a form**. The user does the translating, and only someone who knows the product's structure can finish the job.

v70 THE TEAM, AFTER OUR MEETING

Chat beside a canvas

The right bet, and the one everything since is built on: talk to Ray, watch the flow appear next to you. Staged journey, per-node editing, document upload, a status line for Ray.

⚠ But the system came back around it — an **8-icon rail**, dense side panels, and “Creation · step 3 of 5”. A wizard with a chat window in it.

v3 THIS PROTOTYPE

A playbook you read

One surface. Ray writes the job as numbered plain sentences, each tied to a policy clause — read it like a new hire's manual. Four human stages instead of wizard steps. Edit by tapping a step or by telling Ray.

✓ Nothing to learn before you start: the artefact **is** the interface, and the two questions answer themselves.

WHY A CONVERSATION, NOT A FORM

Six rules behind the chat approach

Every screen in the prototype can be traced back to one of these.

Conversation, not configuration

If you can say it to a colleague, you can say it to Ray — typed or spoken. No rule syntax, no builder grammar to learn.

Trust is built in, not bolted on

Ray practises, proves itself and gets signed before it touches real work. Every change is tracked and auditable.

Less effort, same control

You choose how often Ray checks with you — every step, only what matters, or Autopilot. It never changes what you must sign.

Anyone can build it

A compliance lead, not an engineer, teaches Ray the job. That is what collapses time to value.

Show, don't describe

The case is a map you play through time; the playbook is numbered sentences tied to policy clauses. Understanding is visual, not textual.

The user shapes the room

Two views on every screen, movable panels, role-based environments, a phone-sized manager pulse — and one click back to default.

THE TEAM'S V70 · AND WHAT I CHANGED IN V3

Same bet, fewer things in the way

TEAM CONCEPT (V70)

Established the direction

- ✓ Chat beside a canvas as the way to build a use case — the core bet, and the right one.
- ✓ A staged journey, per-node editing of the flow, document upload, a status line for Ray.
- — Framed as a **wizard**: “Creation · step 3 of 5”, with hint cards pushing you forward.
- — An **8-icon left rail** and dense side panels — the configuration metaphor returns through the back door.
- — Position is shown **passively**; readiness is a number you have to find and interpret yourself.

MY CONCEPT (V3)

Removed the system around it

- ✓ One surface: a **living playbook** you read like a new hire's manual, plus Ray in a column you can hide.
- ✓ Not five wizard steps but **four human stages** — Build, Train, Prove, Operate — the way you onboard a person.
- ✓ No icon rail. Controls split by job: how often Ray asks, and everything else under one ⋯ menu.
- ✓ **Two questions answered on demand** instead of status to decipher.
- ✓ Edit by **tap or by talk**, tracked as versioned changes until you sign them.

WHY IT IS BETTER

Six advantages of this approach

1 Two questions, always answered

Is Ray ready? and **Where am I?** sit permanently above the work. One click gives the verdict, the targets against their thresholds, and the one thing that closes the gap — or the stage, the signed version and who acts next.

2 Onboarding, not deployment

Build → Train → Prove → Operate. Ray practises on alerts your analysts already closed and shadows a senior on live ones before it works alone. Nobody has to trust a black box.

3 The policy is the product

Every step quotes the clause it came from. Reviewers read sentences, not a flow chart — and the model-risk document and analyst guide fall out of it automatically.

4 Autopilot with real brakes

Ray can do the entire job and stop only at the signature gates. The same product serves a hands-on lead and a lead with no time — without two different UIs.

5 Two ways in, for two kinds of people

Speak to Ray, or tap the step and edit the fields. Voice-first for speed, direct editing for control — and the microphone sits in every chat field.

6 It explains itself

A guided walkthrough covers the whole flow on first use and reopens from the top bar any time, and each screen answers for itself — the two questions in Studio, the timeline in a case. A new user is productive without training.

THE TWO QUESTIONS, CONCRETELY

Nobody gets lost in the system

IS RAY READY?

Not yet — 87% against a 90% target

Every target with its threshold, met or below. Then the one action that closes the gap: **“3 suggestions would close it → Show me”**. It reads differently at every stage — untested, practising, shadowing day 9 of 28, live and healthy.

WHERE AM I?

Train · waiting on Dana

The four stages with what is complete, what was skipped and where you are. Which version is current, which is signed, who acts next — and what Ray did while you were away. Ask it out loud and Ray answers in words too.

“Same content. Same engine. The difference is that the product now answers questions instead of displaying state.”

Which is also what makes it demo-able: a five-minute story instead of a feature tour.

1 sentence
to teach Ray a new job

4 stages
before it touches real work

2 questions
that replace reading the system

0 rule syntax
for the person who owns the policy